v9.0.900

Higher-order Functions, CSC430, Winter 2026

back to course homepage

1 Goal

Implement an interpreter for a higher-order language including booleans, strings, and local variable bindings.

2 Purpose

By implementing the core of a language with higher-order functions, you will hopefully be more able to comprehend, formulate, and reason about uses of higher-order functions for the remainder of your programming career. Wow!

Secondarily, this stripped-down language should help you to see how nearly all other language features in modern languages are extensions of a core language based on higher-order functions, and not much else. Awesome!

3 Guidelines

For this and all remaining assignments, every function you develop must come with the following things:

- A commented header line that expresses the result of the function in terms of its inputs, written in English. Be as precise as you can within the space of a line or two.
- A type declaration (possibly inline), specifying the input and output types.
- Test cases. A function without test cases is incomplete. Write the test cases first, please.

For this assignment, you must develop your solutions using the `typed/racket` language. If you haven't seen them, you might be interested in these Hints on Using Typed Racket in CPE 430.

Your test cases must use the `check-equal?`, `check-=`, or `check-exn` forms.

Your solution should take the form of a single file.

Hand in your solution using the handin server. For help with the handin server, please see the course web page.

3.1 Handling Errors

All of your error messages must contain the string `"SZMX"`. Essentially, this allows my test cases to distinguish errors correctly signaled by your implementation from errors *in* your implementation. To be more specific: any error message that doesn't contain the string `"SZMX"` will be considered to be an error in your implementation.

Additionally, your error messages should be actually helpful. Since you are the primary consumer of your own error messages, making these error messages good in the first place should reduce your overall development time. There are two parts to this: first, the error message should include text that actually indicates what the programmer did wrong. Second, include the text of the user's program, so they (actually *you*) can figure out how to fix it. See lab 3 for an example of how to do this. (Apologies in advance if I renumber the labs and fail to update this paragraph....)

3.2 Progress Toward Goal comment

Graders are happier when they know what to expect. Your final submission should start with a short one- or two-line comment indicating how far you got through the project. Ideally, this would just be: "Full project implemented." But if you only implemented, say, squazz and blotz, and didn't get to frob or dringo, please indicate this in the comment, so that we don't spend all our time searching for bits that aren't there.

4 Mutation

There's no need for mutation in any of the first five assignments in this class. Don't mutate bindings, and don't mutate structure fields. You don't have to use hash tables at all, but if you do use hash tables use immutable hash tables only; no `hash-set!`.

5 Extended Interpreter

Write a parser and interpreter for the language we've discussed in class including higher-order functions and environments, extended with the language features described below. Your interpreter should have eager application semantics and use environments. Call the new language SZMX4 .

5.1 Syntax of SZMX4

The concrete syntax of the SZMX4 language with these additional features can be captured with the following EBNF:

```
‹expr› ::= ‹num›
         | ‹id›
         | ‹string›
         | { if ‹expr› ‹expr› ‹expr› }
         | { let { [‹id›=‹expr›]* } in ‹expr› }
         | { fun ( ‹id›* ) => ‹expr› }
```

```
        |  { ‹expr› ‹expr›* }
```

... where an `id` is not `if`, `=`, `let`, `fun`, or `in`.

5.1.1 Syntax Notes:

The “binop" rule is gone, because the primitives are now values.

5.2 Values

The SZMX4 language has a variety of different kinds of values:

- Real numbers,
- Booleans,
- Strings,
- Closures, and
- Primitive Operators, listed below.

Note that in this language, all primitive operators are values, and can be passed and applied like user-defined functions.

5.3 Top-level Env

In order to bind identifiers like `true` and `+` to their corresponding values, we need a top-level environment with these bindings. Note that it’s totally legal to shadow these bindings.

Since these primitives are now no longer part of the grammar, we need to specify their arities explicitly, using a kind of type-like syntax. Note that these are the "types" as they might appear in the SZMX4 manual; this does not say anything about how the language implementor (you) should represent them.

`(+ a b) → real` procedure

  a : real
  b : real

Compute a+b. Signal an error if either *a* or *b* is not a number.

`(- a b) → real` procedure

  a : real
  b : real

Compute a-b. Signal an error if either *a* or *b* is not a number.

```
(* a b) → real                                    procedure
  a : real
  b : real
```

Compute a*b. Signal an error if either *a* or *b* is not a number.

```
(/ a b) → real                                    procedure
  a : real
  b : real
```

If b is not zero, compute a/b. Signal an error if either *a* or *b* is not a number.

```
(<= a b) → boolean                                procedure
  a : real
  b : real
```

Return true if a is less than or equal to b. Signal an error if either *a* or *b* is not a number.

```
(substring s start stop) → string                 procedure
  s : string
  start : real
  stop : real
```

Return a new string containing a substring of the given one, including the character at index *start* up to (but not including) the character at index *stop*. Signal an error if *s* is not a string, and if either *start* or *stop* is not a natural, and if either is out of range for the length of the string, and if *stop* is before *start* (though it's fine for them to be equal).

```
(strlen s) → real                                 procedure
  s : string
```

Return the length of the given string. Signal an error if *s* is not a string.

```
(equal? a b) → boolean                            procedure
  a : any
  b : any
```

Return true if neither value is a closure or a primitive operator and the two values are equal. This function never signals an error, unless it gets the wrong number of arguments. *Note: do not use racket's `eq?` function here or anywhere else in the course.*

```
true : boolean                                    value
```

the literal boolean representing true.

false : boolean value

the literal boolean representing false.

(**error** *v*) → nothing procedure
  v : any

Halt the program, signal an error containing the string "user-error" and the serialization of the given value. (Tip: in Typed Racket, the type containing no values is called `Nothing`.)

5.4 Serialize

The `serialize` function should accept any SZMX4 value, and return a string. I'll be using it in testing your function. For numbers, use ~v directly, so that (for instance) the serialization of your representation of `34` would produce the string `"34"`. The serialization of the `true` value should be the string `"true"`, and the serialization of the `false` value should be the string `"false"`. The serialization of strings should include wrapping double-quotes. You can use the racket function ~v for this as well.

Finally, all closures should be serialized as the string `"#<procedure>"`, and primitive operators should be serialized as the string `"#<primop>"`.

5.5 Function Names

They're gone. No more function names. So, for instance,

```
{fun (a b c) => 3}
```

is now a function of three arguments, named `a`, `b`, and `c`.

5.6 Top-Level Wrapping

Also gone. There's no top-level list of functions; a program is just a single expression, to be evaluated in a top-level environment that binds the primitives.

6 Getting it working

What follows is my recommended iterative development strategy; in the following, I try to keep the evaluator and its test cases working at every step, so that you don't spend too much time wandering in the wilderness of broken code.

Before adding conditionals or `var`, you need to adapt your base evaluator so that it works with environments and higher-order functions, and has a `serialize` function that renders values as strings.

First, define the type of environments, then add it to the list of arguments to `interp`, then adapt varrefs/IdCs so they perform lookup. Finally, alter the rule for applications so that it adds things to an empty environment rather than performing substitution. The code should work on a consistent set of test cases all through this transition. Finally, you can remove `subst`.

6.1 Adding Booleans

Next, we're going to add new kinds of values to the language, and we can begin by adding booleans. Once again, iterative development is your friend. First, add a `Value` define-type, initially including only numbers and booleans. Then adapt the evaluator so that it returns a `Value` rather than a number. Define a top-level environment with bindings for `true` and `false`.

Now, add a simple `serialize` function that handles numbers and booleans.

Next, add a call to `serialize` to your `top-interp`.

6.2 Function Values (closures)

Next, add closures to the set of values. This is pretty much straight out of the book.

Extend `serialize` to handle closures.

Once you've got this working, you can try sticking some function values into the initial environment passed to the evaluator. Then, update the `ExprC` definition so that the first position in an application is an arbitrary expression (`ExprC`) rather than a symbol, and update the evaluation rule for application so that the first position is interpreted, rather than just being looked up. At this point, your test cases with function definitions will be ignoring the list of functions, and just looking in the environment for their definitions. Those definitions will all have to be supplied manually by your test cases, though, because you don't have any way of defining them in the code. Now, you can yank the top-level funs argument to the interpreter, because it's not being used.

Next, you need to make it possible to define functions in the program; do this by adding a function literal form (the `lamC` form of the book), and the accompanying parser and interpretation rules. At this point, you should be able to specify test cases that include literal functions, and apply them to arguments.

Get rid of `parse-prog`, and rewrite `top-interp`.

6.3 Dumping binops

Let's get rid of binops. First, we need to add at least one primitive operator. Define a representation for primitive operators, and add it to your set of Values. Add a binding for this binary operator to your top-level environment. Extend `serialize` to handle primitive values.

Next, update your parser to rip out the specialized parsing for binops, and the restrictions on identifiers that prevent them from being operator names.This should consist mostly of deleting

code, and updating test cases.

Finally, rip out your binop code from interp, and add code for `AppC` exprs that distinguishes primitive operators from closures, and handles them correctly.

6.4 Booleans and comparisons

Note that you'll have to update your binops so that they can deal with `Values` returned by `interp` rather than simple numbers (the textbook covers this).

In order to make our real boolean conditionals useful, we want some way to compare numbers. The simplest useful operation is `<=` (using it, we can model all of the other numeric comparison operators). Add the `<=` operator to your set of binops.

In addition, let's add an `equal?` operator. This operator should return `true` if given two booleans with the same value, or two numbers with the same value, or two strings with the same value. If the operands are different kinds of value or if they're both function or primitive values, this operator should return `false` (this is not an error, it just returns false).

Read that previous paragraph again carefully, would you?

6.4.1 Don't use `eq?`

You should not use the function `eq?` to compare strings; it performs a pointer comparison, like java's "==", which can bite you on this assignment. Instead, use the `equal?` function to compare strings.

6.5 Conditionals

Now that we have booleans, we can change the icky awful `ifleq0` into a nice clean `if`.

First, rewrite your test cases so that they use the new boolean-producing operators, and then modify your expression definition to change `ifleq0` into `if`, updating the parser and the evaluator (slightly). The `if` expression should signal an error if its `test` expression does not produce a boolean value.

7 local variables

As you know, local variable definitions are incredibly useful. As Shriram points out, though, you can get them "for free", by desugaring them into applications. Specifically, you can change

```
{let
  {[z = {+ 9 14}]
   [y = 98]}
 in
   {+ z y}}
```

into

```
{{fun (z y) => {+ z y}}
 {+ 9 14}
 98}
```

You must add this `with` syntax to your parser. it will make your test cases read much more nicely.

You must implement `with` using desugaring, but it's up to you whether you do this using a separate `desugar` pass or just handle it in the parser.

Note that the given desugaring pattern means that the scope of the newly bound variables includes only the `with` body, *not* the right-hand-sides of the later bindings.

7.1 Error-checking in Parse

Your parser should detect all terms that fail to match the EBNF, and signal an error. For instance, an `if` that's missing an 'else' clause, or a function definition with two parameters named `x`, should both cause an error.

7.2 Error-checking in Interp

Your interpreter must signal an error for "type-like" errors; application of a non-closure, calling an arithmetic primitive with a non-number, applying a function to the wrong number of arguments, using a conditional with a non-boolean test.

8 Interface

Make sure that you include the following functions, and that they match this interface:

```
(parse s) → ExprC                                    procedure
  s : Sexp
```

Parses an expression.

```
(interp e env) → Value                               procedure
  e : ExprC
  env : Environment
```

Interprets an expression, with a given environment.

```
(top-interp s) → string                              procedure
  s : Sexp
```

Combines parsing and evaluation. Here's the code you should probably use for this function:

```
(define (top-interp [s : Sexp]) : String
  (serialize (interp (parse s) top-env)))
```

Your body can be different if you really want, but the types must match these.